

■ Sujet de l'exercice correspondant au code (Partie 1)

Objectif pédagogique : Réimplémenter manuellement les formules fondamentales vues le matin, afin de bien comprendre leur fonctionnement avant d'utiliser des bibliothèques comme scikit-learn.

Durée indicative : 45 minutes

Consignes : Vous allez coder dans un script Python (ou notebook) les quatre fonctions suivantes, puis les tester sur des exemples simples.

1. Fonction `entropie(proportions)`

Calculez l'entropie de Shannon en bits à partir d'une liste de proportions des classes.

Formule :

$$H = - \sum_i p_i \cdot \log_2(p_i)$$

Rappel :

- Si une proportion vaut 0, on l'ignore (car $0 \times \log_2(0) = 0$ par convention).
- Le logarithme est en base 2, donc le résultat est en bits.

Testez avec :

Entrée	Résultat attendu
[0.5, 0.5]	1.0 bit
[0.8, 0.2]	≈ 0.722 bit
[1.0, 0.0]	0.0 bit

2. Fonction `gain_information(entropie_parent, sous_ensembles)`

Calculez le gain d'information apporté par une division.

Formule :

$$\text{Gain} = H(\text{parent}) - \sum_v (|S_v| / |S|) \cdot H(S_v)$$

Paramètres :

- `entropie_parent` : float, entropie du nœud parent.
- `sous_ensembles` : liste de tuples (taille_du_sous_ensemble, entropie_du_sous_ensemble).

Testez avec l'exemple du dataset météo (10 jours) :

- Entropie initiale = 1.0 bit.
- Caractéristique **Temps** :

Modalité	Nb jours	Répartition	Entropie
Soleil	4	1 Oui / 3 Non	≈ 0.811 bit
Couvert	2	2 Oui / 0 Non	0 bit

Pluie	4	2 Oui / 2 Non	1 bit
-------	---	---------------	-------

Vous devez obtenir un gain d'environ **0.2756 bit**.

3. Fonction `sigmoide(z)`

Implémentez la fonction sigmoïde qui transforme un score z en probabilité comprise entre 0 et 1.

Formule :

$$\sigma(z) = 1 / (1 + e^{-z})$$

Testez avec quelques valeurs :

z	Résultat attendu
-2	≈ 0.119
0	0.5
2	≈ 0.881

Visualisation : tracez la courbe sur l'intervalle $[-10, 10]$ avec matplotlib. Ajoutez une ligne horizontale à 0.5 et une verticale à 0.

4. Fonction `log_loss(y_true, y_pred_proba)`

Calculez la perte logistique (log loss) pour un échantillon.

Formule :

$$LogLoss = - [y \cdot \log(p) + (1 - y) \cdot \log(1 - p)]$$

où y est la vraie étiquette (0 ou 1) et p la probabilité prédite pour la classe 1.

Astuce : ajoutez une petite valeur epsilon (ex. 10^{-15}) pour éviter $\log(0)$.

Testez :

y_true	p	Perte attendue
1	0.9	≈ 0.105
1	0.1	≈ 2.303

5. Mini-exemple de régression logistique (manuel)

À partir des fonctions précédentes, simulez un modèle de régression logistique simple avec une seule variable x .

On donne $\beta_{\text{intercept}} = 0.5$, $\beta_{\text{feature}} = 0.3$.

- Calculez le score $z = \beta_{\text{intercept}} + \beta_{\text{feature}}x$ pour $x = 1$.
- Transformez ce score en probabilité avec la sigmoïde.
- Calculez la log loss correspondante si la vraie étiquette est 1.

Résultat attendu :

z	Probabilité	Log loss
0.8	≈ 0.690	≈ 0.371

■ Livrable attendu

Un script Python commenté contenant :

- Les quatre fonctions demandées.
- Les tests avec affichage des résultats.
- La visualisation de la sigmoïde.
- Une conclusion rapide sur l'utilité de chaque fonction dans la construction d'un arbre de décision et d'une régression logistique.

Remarque : Dans le code fourni, certaines valeurs de l'exemple météo sont inversées (Soleil et Pluie). Corrigez-les pour obtenir le bon gain d'information (0.2756 bit).